Agenda

Angular Fundamentals

- Data Binding
- Directives
- Pipes
- Routing
- Route Parameters
- Lazy Loading

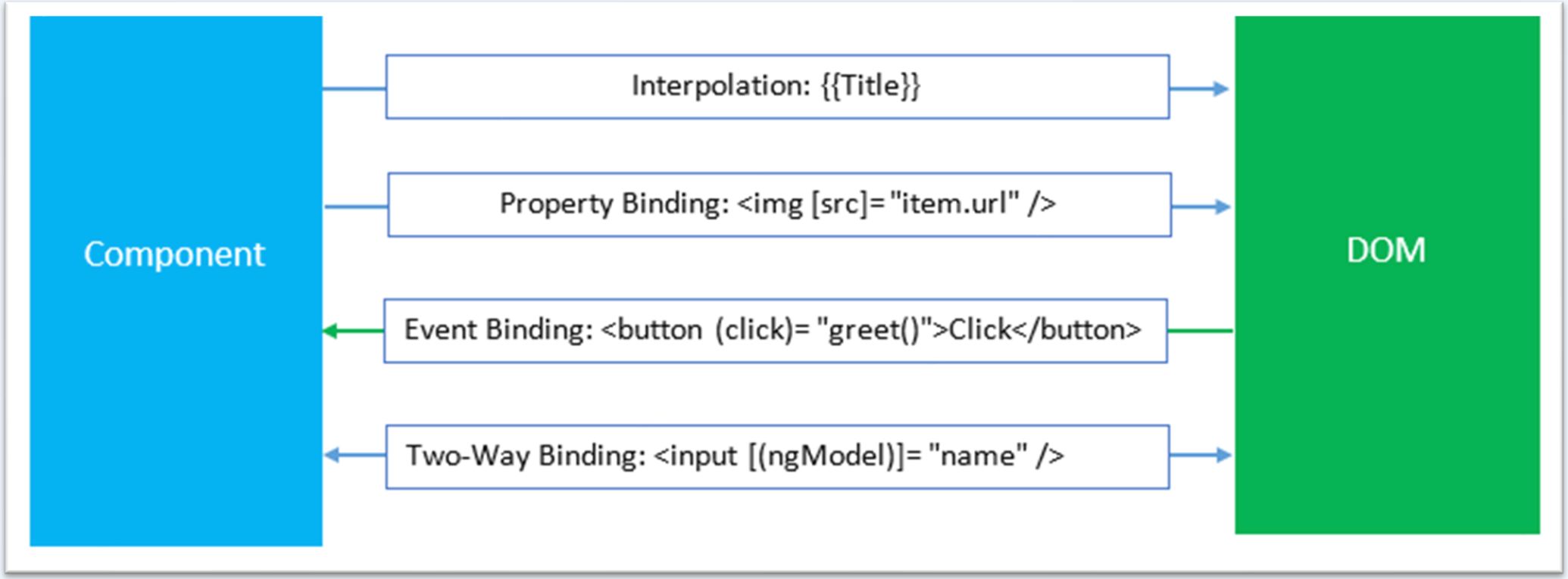
Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Databinding

- A mechanism for binding data values to HTML elements and turning user activities into actions
- Responsible for data value updates
- Support the following Databinding:
  - Interpolation
  - Property Binding
  - Two-way Binding
  - Event Binding

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Data Binding Syntax



Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Directives

- Used to extend the power of existing Html markup
- Change the appearance or behavior of a DOM element
- There are four kinds of directives in Angular2
  - Components – A directive with a template. @Component decorator extends the @Directive decorator with template-oriented features
  - Structural directives – Alter layout by adding, removing, and replacing elements in DOM Eg. *ngIf, *ngFor, *ngSwitch
  - Attribute directives – Alter the appearance or behavior of an existing element. Eg. ngModel, ngStyle, ngClass
  - Custom directives – Your own directive

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Custom Directive

- @Directive decorator is used to create a custom directive
- Custom directive can be used as an attribute or class

```
@Directive({
  selector: '[appHighlight],.appHighlight'
})
export class HighlightDirective {
  constructor(private el: ElementRef) {
    this.color = 'black';
  }
  @HostBinding('style.color') color: string;
  @HostListener('mouseover') f1() {
    this.color = 'red';
    this.el.nativeElement.style.backgroundColor = 'yellow';
  }
  @HostListener('mouseleave') f2() {
    this.color = 'black';
    this.el.nativeElement.style.backgroundColor = '';
  }
  @HostListener('click') f3() {
    alert(this.el.nativeElement.innerHTML);
  }
}
```

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Pipes

- Used to transform bound properties before displaying
- Angular built-In Pipes are :
  - Lowercase
  - Uppercase
  - Date
  - Currency
  - Json
  - Decimal
  - Async etc.

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Custom Pipe

- A custom pipe can be used for applying you own logic to transform text
- @Pipe decorator is used to create a custom pipe

```
@Pipe({
  name: 'reverse'
})
export class ReversePipe implements PipeTransform {
  transform(value: any, args?: any): any {
    let result = '';
    for (let i = 0; i < value.length; i++) {
      result = value[i] + result;
    }
    return result;
  }
}
```

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Routing

- Serve webpages based upon the requested url
- In Angular applications URLs are not served from the server and never reload the page, every time a user requests for a new page
- The URLs are strictly local in the browser and serve the page from local browser itself
- Every time when you request a Url, the Angular router navigates to the new component and renders its template and updates the history and URL

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Routing Configuration

- **Routes** describe the routes

```
const routes: Routes = [
    { path: '', component: DatabindingComponent },
    { path: 'about/:id', component: AboutComponent },
    { path: 'notfound', component: NotfoundComponent },
    { path: '**', redirectTo: 'notfound' }
];
```

- **RouterOutlet** is where the router renders the component

```
<div class="container">
  <router-outlet></router-outlet>
</div>
```

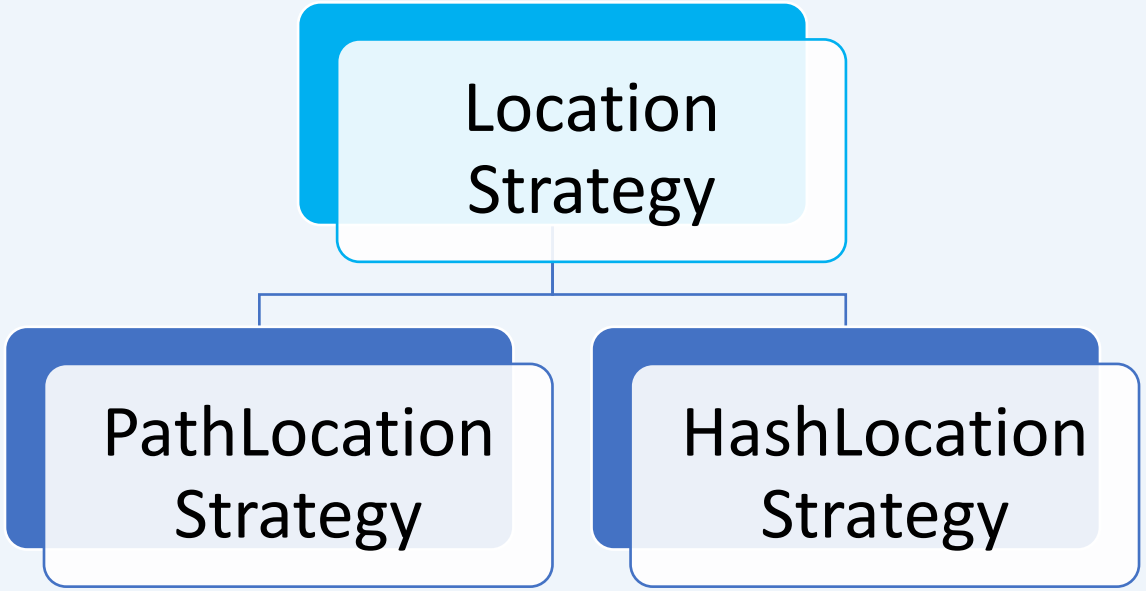
- **RouterLink** a link to a route name

```
<a [routerLink]="['/about','4']">About</a>
```

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Location Strategies

- Defines how the URL/Request is being resolved
- Also determines how your URL will look like
- Angular supports two Location Strategies



Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

PathLocation Strategy

- Here, URL looks like http://localhost:4200/product
- Supports Server-Side Rendering
- SEO Friendly
- Older Browsers don’t support
- For Configuration use
  - <base href="/"> at main page
  - RouterModule.forRoot(appRoutes) in routing module

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

HashLocation Strategy

- Here, URL looks like *http://localhost:4200/#/product*
- Supported by all browsers
- For Configuration use
  - <base href="/"> at main page
  - RouterModule.forRoot(appRoutes, {useHash: true}) in routing module

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Route Parameters

- Used to pass data through route paths
- Mainly there are the following ways :
  - Required Parameters
  - Optional Parameters
  - Query Parameters

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Required Parameters

- Setup

```
{ path: 'product/:id', component: ProductComponent }
```

- Activate

```
<a [routerLink]="['/product',4]">Product</a>
this.router.navigate(['product', 4]);
```

- Retrieve

```
this.route.params.subscribe((p) => {
  this.id = p.id;
});
```

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Optional Parameters

- Setup

```
{ path: 'product', component: ProductComponent }
```

- Activate

```
<a [routerLink]="['/product',{id:1,name:'books'}]">Product</a>
```

```
this.router.navigate(['product', { id: 1, name: 'books' }]);
```

- Retrieve

```
this.route.params.subscribe((p) => {
  this.id = p.id;
  this.name = p.name;
});
```

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Query Parameters

- Setup

```
{ path: 'product', component: ProductComponent }
```

- Activate

```
<a [routerLink]="['/product']" [queryParams]="{id:1,name:'books'}">Product</a>
```

```
this.router.navigate(['/product'], { queryParams: { id: 3, book: 'Angular' } });
```

- Retrieve

```
this.route.queryParams.subscribe((p) => {
  this.id = p.id;
  this.name = p.name;
});
```

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Child/Nested Routes

- Child/Nested routes is a powerful feature in Angular router to render child/nested components using route
- Children option is used to define child/nested routes

```
const routes: Routes = [
    { path: '', component: DatabindingComponent },
    {
        path: 'about', component: AboutComponent, children: [
            { path: 'profile', component: ProfileComponent } ]
    }
];
```

Setup Child Routes

```
<a [routerLink]="['profile']">Show Profile</a>
<router-outlet></router-outlet>
```

Activate Child Routes

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

Lazy Loading

- Lazy loading is a technique that allows you to load Angular components asynchronously when a specific route is activated
- Improves initial performance during the initial load, especially if you have many components with complex routing
- Lazy module loading is useful for developing more than one area(modules) in an application

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

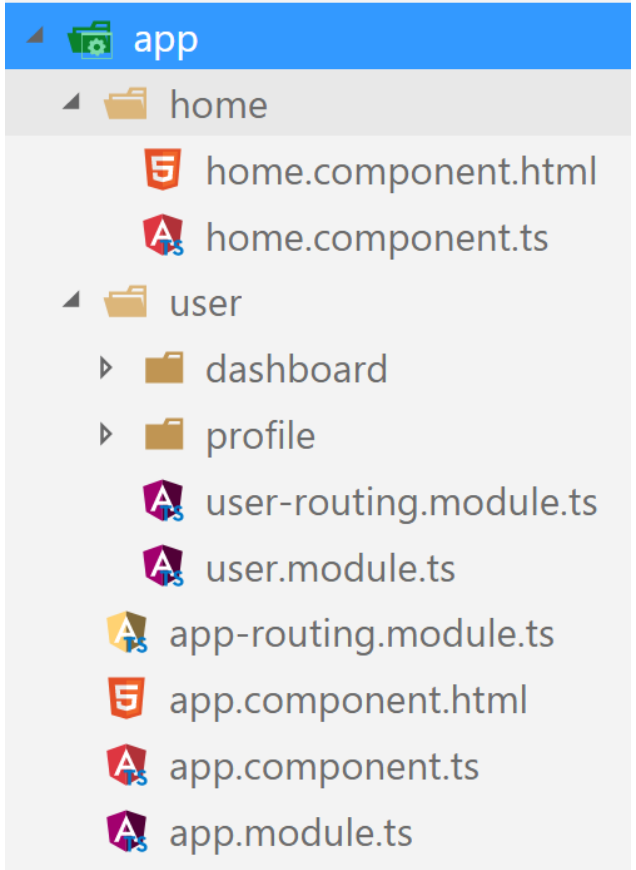
Configuring Lazy Loading

Step 1 : Use loadChildren

```
{ path: 'admin', loadChildren: () => import('./admin/admin.module').then(mod =>
mod.AdminModule) }
```

Step 2 : Define routes for lazy load module

```
const routes: Routes = [
  { path: '', component: DashboardComponent }
];
@NgModule({
  imports: [RouterModule.forChild(routes)],
  exports: [RouterModule]
})
export class UserRoutingModule { }
```



Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

THANKS

Any Questions?

Copyright Dot Net Tricks Innovation Pvt. Ltd. | All rights Reserved.

ScholarHat